Departamento de Ingeniería de Sistemas
Arquitectura de Computadores y Laboratorio – 2026-2
Laboratorio No. 3
Diseño y verificación del procesador monociclo

UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

Fecha de entrega: 23:59 horas del 16 de noviembre de 2026

1. Objetivo

- Comprender los requerimientos de una versión reducida de la arquitectura MIPS de 32 bits para realizar su implementación en forma de procesador monociclo (ruta de datos y unidad de control).
- Codificar, ensamblar y simular un programa de prueba para verificar el comportamiento correcto del procesador implementado.
- Emplear herramientas de software para el diseño y la simulación de computadores digitales.

2. Descripción

En esta práctica cada equipo realizará una implementación como procesador monociclo de una versión reducida de la arquitectura MIPS32 en la herramienta Logisim Evolution v.3.8.0. Partiendo de la implementación básica mostrada en la Figura 1, cada equipo de trabajo deberá extender el repertorio de instrucciones soportadas en hardware. Una vez implementadas la ruta de datos y la unidad de control, el procesador será puesto a prueba para verificar el correcto funcionamiento de todas las instrucciones incluidas, y luego será usado para resolver un problema en particular asignado a cada equipo de trabajo.

El procesador debe estar en capacidad de ejecutar las siguientes instrucciones: **`load word`**, **`store word`**, **`add`**, **`sub`**, **`and`**, **`or`**, **`nor`**, **`set-on-less-than`**, **`branch on equal`** y **`jump`**, siguiendo una organización como la que se muestra en la Figura 1. Luego, cada equipo de trabajo debe agregar las instrucciones **`jump-and-link`** y **`jump-register`**, y dos más como se explica en la Sección 3 de esta guía. Los formatos de instrucción con sus códigos de operación, identificadores de registros y valores del campo función del formato R se deben ajustar a los definidos en el documento MIPS Reference Data. El control de la ruta de datos se debe realizar con los módulos "Control" y "ALU control" incluidos en la Figura 1, aplicando técnicas de diseño combinacional estudiadas previamente en el curso.

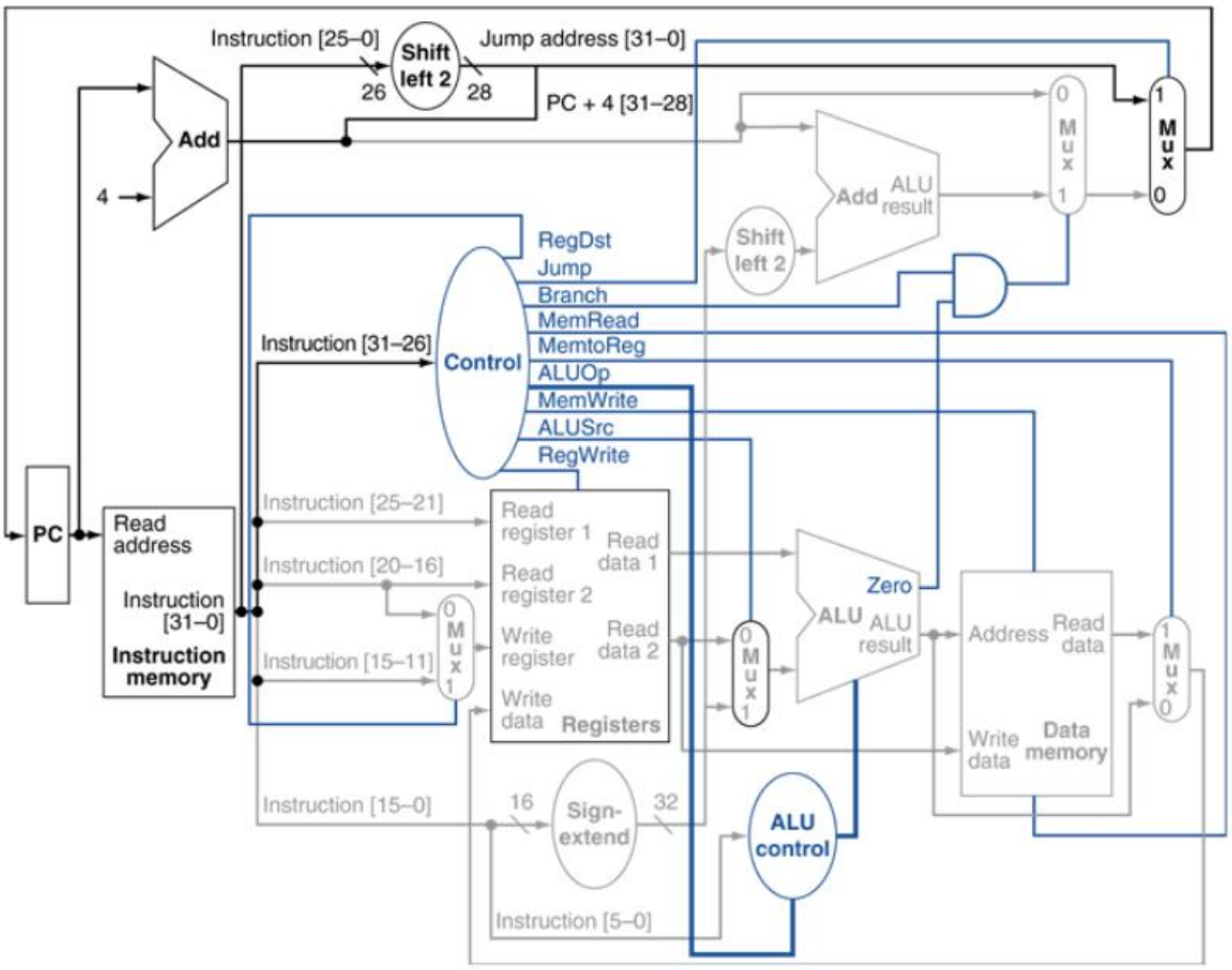


Figura 1. Implementación monociclo de la arquitectura MIPS capaz de ejecutar instrucciones lw, sw, add, sub, and, or, nor, slt, beq y jump.

El diseño del banco de registros se debe realizar siguiendo un estilo jerárquico, partiendo de un registro de 32 bits disponible en la biblioteca de componentes de **Logisim Evolution**, que será instanciado tantas veces como sea necesario para conformar el banco. El banco de registros debe cumplir con las siguientes condiciones: (i) la lectura de los registros debe ser asíncrona, mientras que la escritura debe ser síncrona; y (ii) de acuerdo con la arquitectura del conjunto de instrucciones MIPS de 32 bits, el registro **`$zero`** siempre debe entregar el valor cero cuando sea accedido en modo de lectura, y su valor no puede ser modificado mediante una operación de escritura. La ALU de 32 bits corresponderá al componente perfeccionado elaborado en el primer laboratorio del curso. La biblioteca de componentes de **Logisim Evolution** podrá ser empleada con total libertad para el diseño de los demás elementos del procesador.

Durante el proceso de diseño e implementación del procesador pueden presentarse diversas situaciones ante las cuales cada equipo debe tomar decisiones. En todos los casos, las decisiones de diseño deben estar ampliamente explicadas y justificadas.

3. Instrucciones adicionales por implementar y problema a resolver

Las dos instrucciones por agregar y el problema a resolver mediante programación en bajo nivel para su ejecución en el procesador implementado por parte de cada equipo se muestran en la Tabla 1. La asignación se calcula como (*Número del equipo* % 8). Los valores de la señal de control **`ALUOp`** quedan a discreción de los equipos de trabajo.

La solución del problema asignado debe ser codificada con las instrucciones soportadas por el procesador implementado. Los requisitos a los que se debe ajustar la codificación de la solución son los siguientes:

- Programación basada en el uso de procedimientos: el código que resuelve el problema debe ser estructurado, de manera que es obligatorio emplear las instrucciones **`jal`** y **`jr`**.
- Seguir la convención para el uso de registros MIPS.

Durante la ejecución del código que resuelve el problema asignado, la visualización de los resultados que produce se deben realizar a través de los elementos de almacenamiento disponibles (banco de registros y memoria de datos), ante la carencia de un sistema de entrada y salida en el procesador. Se valorará la creatividad y elegancia en la programación. Se recomienda el uso de la herramienta **MARS** para la escritura del código ensamblador, su depuración y la obtención del código binario correspondiente.

Los equipos de trabajo estarán en libertad de implementar más instrucciones de las que son exigidas. La correcta ejecución de todas las instrucciones implementadas debe ser evidenciada en la entrega.

Tabla 1. Asignación de instrucciones adicionales y problema

Asignación	Instrucciones adicionales	Problema

0	`Shift left logical`
	
	`Load upper immediate`
		El programa debe tomar un vector de diez números enteros y ordenarlo de manera ascendente, indicando cuántos elementos son negativos y cuántos son positivos.

1	`Load halfword unsigned`
	
	`And immediate`
		El programa debe tomar un vector de diez números enteros y ordenarlo de manera descendente, indicando cuántos elementos son pares y cuántos son impares.

2	`Load byte unsigned`
	
	`Branch on not equal`
		El programa debe tomar un vector de diez números enteros y determinar cuáles son el mayor y el menor número almacenados.

3	`Add immediate`
	
	`Load halfword`
		El programa debe tomar una matriz de 3×3 números enteros y determinar cuáles son el mayor y el menor número almacenado en cada fila de la matriz.

4	`Shift right logical`
	
	`Load byte`
		El programa debe tomar una matriz de 3×3 números enteros y determinar cuántas veces está presente el valor cero (0) en cada columna de la matriz.

5	`Or immediate`
	
	`Load upper immediate`
		El programa debe tomar un vector de diez números enteros y rotarlo hacia la derecha un número de posiciones determinado. Este número será fijado en una posición de memoria antes de la ejecución.

6	`Set on less than immediate`
	
	`And immediate`
		El programa debe tomar un vector de diez números enteros y rotarlo hacia la izquierda un número de posiciones determinado. Este número será fijado en una posición de memoria antes de la ejecución.

7	`Load byte`
	
	`Add immediate`
		El programa debe calcular los primeros diez elementos de la serie $S(n) = 2S(n-1) + S(n-2)$, con $S(0) = 0$ y $S(1) = 1$.

4. Informe

Cada equipo de trabajo debe realizar un informe escrito que incluya una descripción completa del proceso de ampliación y enriquecimiento del procesador monociclo (ruta de datos y unidades de control) presentado en la Figura 1, tal como lo plantea la guía del laboratorio. Esta parte del informe debe contener los esquemáticos circuitales y la explicación y justificación de las decisiones de diseño tomadas durante el desarrollo de la práctica. Asimismo, debe evidenciar el proceso de verificación de la ejecución correcta de todas las instrucciones que soporta el procesador. También debe incluir una descripción de alto nivel (mediante pseudocódigo, diagrama de flujo, etc.) del programa implementado que da solución al problema asignado, así como su implementación en lenguaje ensamblador y su respectivo código de máquina. El informe debe contener resultados de ejecución, y las observaciones y conclusiones pertinentes del trabajo. Este documento debe estar en formato **PDF** y ser subido a la plataforma antes del cierre del plazo de entrega, con los siguientes archivos como adjuntos: diseño en **Logisim Evolution**, código ensamblador MIPS del problema asignado, código de máquina MIPS equivalente y, si es necesario, contenido de la

memoria de datos previo a la ejecución. Todos los archivos, sin excepción, deben ser subidos a la plataforma. El informe tiene un peso del 30% en la calificación global de la práctica.

5. Sustentación

   Ambos miembros del equipo de trabajo deben demostrar un dominio completo del desarrollo de la práctica. Esta demostración será realizada, para los equipos que tengan un **identificador impar**, mediante un video con una duración máxima de ocho minutos, en el que intervengan de manera equitativa ambos miembros del equipo. La entrega del video se debe realizar como un enlace de acceso al mismo incluido en el informe, y no como un archivo adjunto. Por su parte, los equipos con **identificador par** deberán realizar esta demostración de manera síncrona ante el profesor o el monitor del curso, en horario y medio a convenir, durante la semana posterior a la entrega. La calificación de la sustentación tiene un peso del 70% en la calificación global de la práctica, e incluye también la verificación del funcionamiento de la solución entregada.